

第50回技能五輪全国大会 「電子機器組立て」職種

競技Ⅰ

別添資料

1. USB 配布データ
2. 回路設計報告書(設計シート)①～③
3. モータコントローラの使い方
4. G&A Terminal の動作について
5. G&A Terminal 回路
 - グラフィック LCD & 警報ターミナル回路図
 - Graphic LCD & Alarm Terminal Board 基板図
6. G&A Terminal ソースプログラム
 - lcm240_v8.c
 - 関数一覧表
7. ギブアップ宣言用紙

USB 配布データ

配布した USB メモリのフォルダを図 1 に示します。ファイル名やフォルダ名の xx は各自の選手番号 (2 桁) に、name は各自の名前 (ローマ字で) に変更してください。

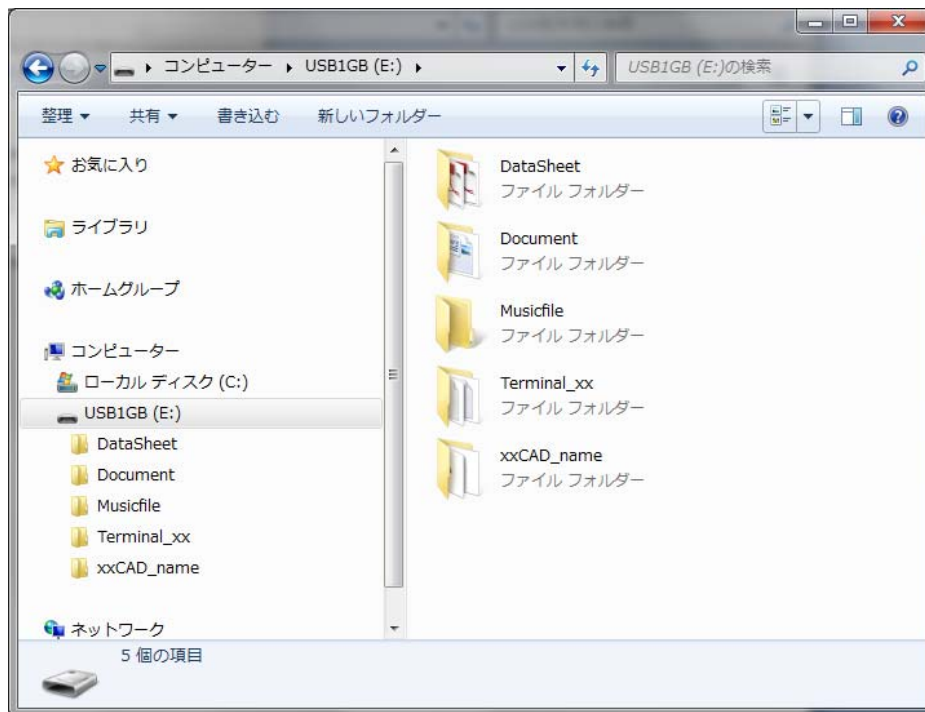


図 1 USB メモリにある 5 のフォルダ

(1) Datasheet フォルダ

設計・試作課題で使用する部品のデータシートが pdf 形式のファイルで格納されています。必要なファイルは印刷してください。

(2) Document フォルダ

回路設計報告書やプログラム設計シートなど、Word ファイルと Excel ファイルが格納されています。これらは印刷物で提出してください。

(3) xxCAD_name フォルダ ≫ フォルダ名変更

Altium Designer 技能五輪用テンプレートファイルが格納されています。
“Template&Library20120913” に 3 つのファイルを追加しています。

- gorin2012.SchLib
- gorin2012.PcbLib
- gorin_121027_ICB96.PcbDoc

(4) Terminal_xx フォルダ ≫ フォルダ名変更

G&A Terminal のプログラミング設計課題の “kadai_xx.c” と “kadai_xx.h” が格納されている “terminal_xx” のフォルダです。

回路設計報告書（設計シート）①

●モータ駆動回路の設計

競技者番号：

競技者氏名：

回路設計報告書（設計シート）②

●各素子の値

競技者番号：

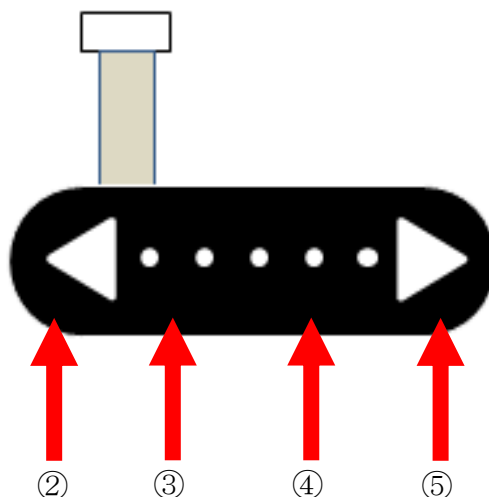
競技者氏名：

回路設計報告書（設計シート）③

●動作確認

タッチセンサ操作モードでタッチセンサを押したときの，LCD に表示される回転方向，回転数を書きなさい．（変動が大きいので大体の値でよい）

① 起動しただけ



タッチセンサの場所	回転方向の表示 R/L	回転数の表示 rpm
①		
②		
③		
④		
⑤		

競技者番号：

競技者氏名：

モータコントローラの使い方

モータコントローラの構成

モータの回転方向、回転数の制御をタッチセンサ、及び Graphic LCD & Alarm Signal ターミナル（以下、ターミナル）からできます。また、回転方向と回転数は随時 LCD、及びターミナルに表示され、現在の回転情報をモニタすることができます。図 1 にモータコントローラの構成図を示します。

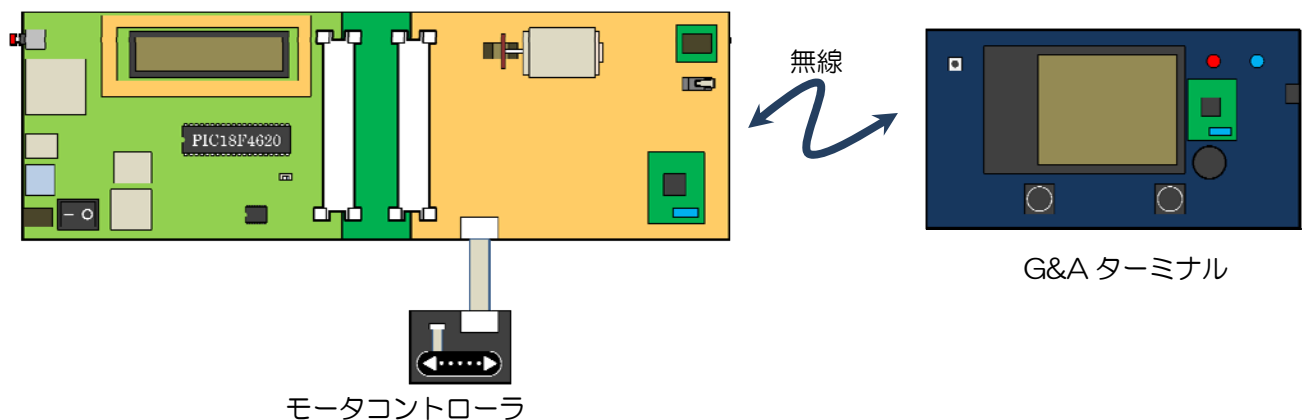


図 1 モータコントローラの構成図（設計・試作基板）

このモータコントローラの競技仕様は、以下の通りです。

- モータコントローラはカップリングボードを使用し、CPU ボードⅢ（以下、CPU ボード）と接続します。
- モータの電源だけ外付けの+5V を使用し、それ以外は、バスラインの+5V を使用します。
- モータコントローラの制御はタッチセンサモジュール及びターミナルを用います。
- モータコントローラは、ターミナルと ZigBee モジュールにより無線で通信します。
- ターミナルからの制御は、タッチパネルの入力方法と、加速度センサを用いた入力方法があります。
- ターミナルからも現在のモータの回転方向、回転数をモニタすることができます。

動作モード

モータコントローラは、図 2 のように動作します。タッチセンサ操作モードと遠隔操作モードの 2 つの動作モードがあり、ターミナルとの連動でモードが切り替わります。

- ・ CPU ボードの電源、モータコントローラの電源を投入します。
- ・ モータコントローラとターミナル間で ZigBee が通信している時は、赤色 LED は点灯し、通信できていない時は、点滅します。

(1) タッチセンサモード

- ・ 電源投入時はタッチセンサ操作モードで起動します。
- ・ タッチセンサを強く押すと、モータの回転方向、回転数がロック状態になり、タッチセンサに触れても、変化しません。
- ・ もう一度タッチセンサを強く押すと、ロック状態は解除されます。

(2) 遠隔操作モード

- ・ ターミナルから信号が入ると、遠隔操作モードに移行します。
- ・ ターミナルの遠隔操作モードにはマトリックスキー入力モード、加速度センサ入力モードがあります。
- ・ 遠隔操作モードに切り替わるのは以下の通りです。
 - マトリックスキー入力モード：「RUN」キーを押したとき。
 - 加速度センサ入力モード：随時通信しています。
- ・ タッチセンサを押すとタッチセンサモードに移行します。
- ・ 各モードで、実際のモータの回転方向、回転数をターミナルに送信し、モニタすることができます。

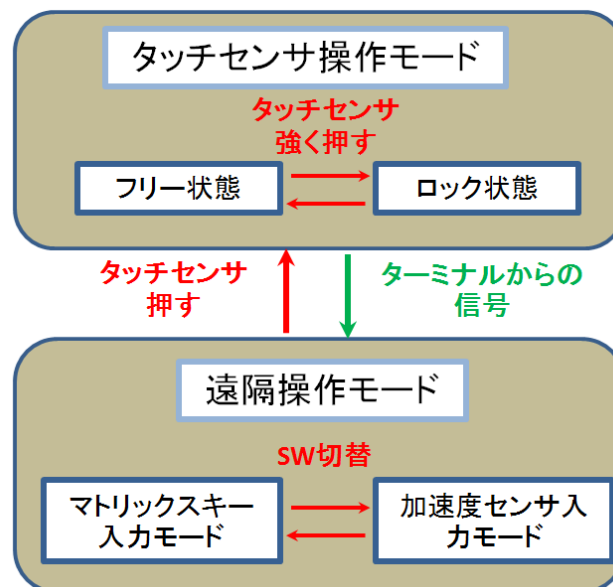
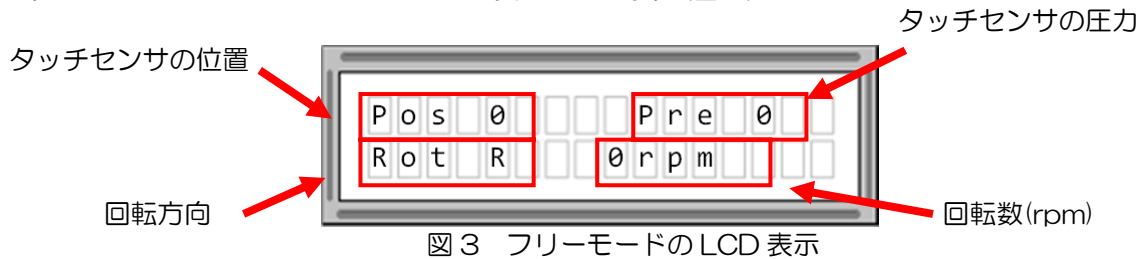


図 2 モータコントロールの動作モード

タッチセンサ操作モード

このモードはタッチセンサでモータの回転方向、回転数を変えることができます。

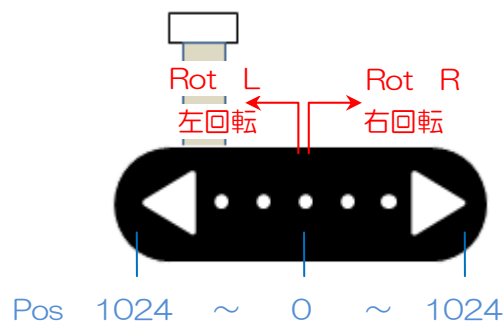
1. CPU ボードの LCD に以下のように表示されます。(図 3)



2. タッチセンサの操作について以下に示します。

(1) タッチセンサの位置

- ・中央部分から右側を押すと、モータが右回転 (Rot R)、左側を押すとモータが左回転 (Rot L) します。(図 4)
- ・中央から端の部分を押すに従い、Pos の値が大きくなり、回転数が多くなります。



(2) タッチセンサの圧力

- ・タッチセンサの押す力が増すに従い、Pre が 0 から上昇します。
- ・Pre の値が 100 を超えると、ロック状態に移行します。

3. ロック状態

- ・タッチセンサを押したとき Pre が 100 を超えると、ロック状態になります。(図 5)
- ・ロック状態は、モータの回転方向、回転数が固定され、タッチセンサに触れても回転方向、回転数が変化しません。
- ・このとき、タッチセンサに入力をして、モータの回転方向、回転数は変化しません。
- ・もう一度、Pre が 100 を超えたとき、フリー状態に戻ります。



4. 右回転／左回転の切り替え

- ・ 右回転から左回転，また，左回転から右回転と回転方向が切り替わる際，回路に流れる貫通電流が流れる可能性があります．そこで，切り替わる時に回転を一度止めて方向を切り替えるプログラムをしています．

そのため，回転方向が切り替わる際，1.3 秒程度の待機時間を設けています．

遠隔操作モード

このモードはターミナルから，モータの回転方向，回転数を変えることができます．

1. CPU ボードの LCD に以下のように表示されます．（図 6）

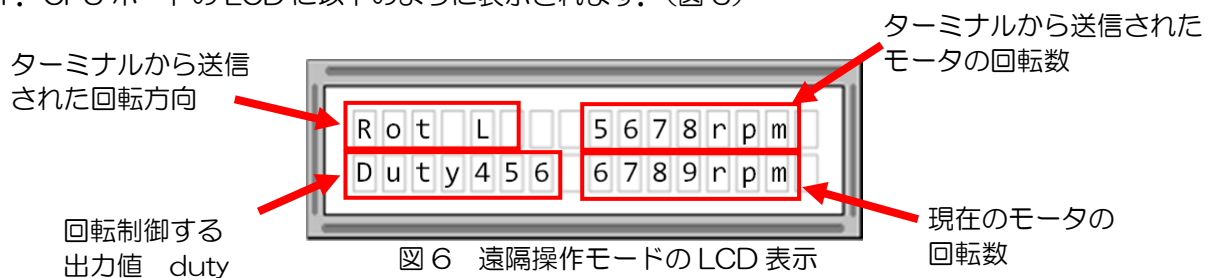


図 6 遠隔操作モードの LCD 表示

2. ターミナルの2つのモード（マトリックスキー入力モード，加速度センサ入力モード）のどちらでも図 6 のように同じ表示となります．
3. タッチセンサを押すとタッチセンサ操作モードに戻ります．

（注意）加速度センサ入力モードのとき，タッチセンサ操作モードに一瞬戻りますが，常にモータコントローラにデータを送信しているため，すぐに遠隔操作モード（加速度センサ入力）に戻ります．

G&A Terminal の動作について

(1) G&A Terminal の動作モード

現在の G&A Terminal は、図 1 のように動作します。初期設定モードと動作モードの 2 つのモードがあります。初期設定モードは、プログラムを書き込み直した場合は、その都度、初期設定を行う必要があります。初期設定モードから動作モードへの切り替えは、図 2 の G&A Terminal の赤丸のディスプレイボタンをタッチペンでタッチします。このボタンを“ディスプレイボタン”と呼ぶことにします。

動作モードには、マトリクスキー入力モードと加速度センサ入力モードの 2 つのモードがあり、各モードの切り替えは、図 2 の G&A Terminal の黄丸の L ボタンスイッチを押します。G&A Terminal の電源を投入すると、図 3 のようなマトリクスキー入力モードが表示されます。

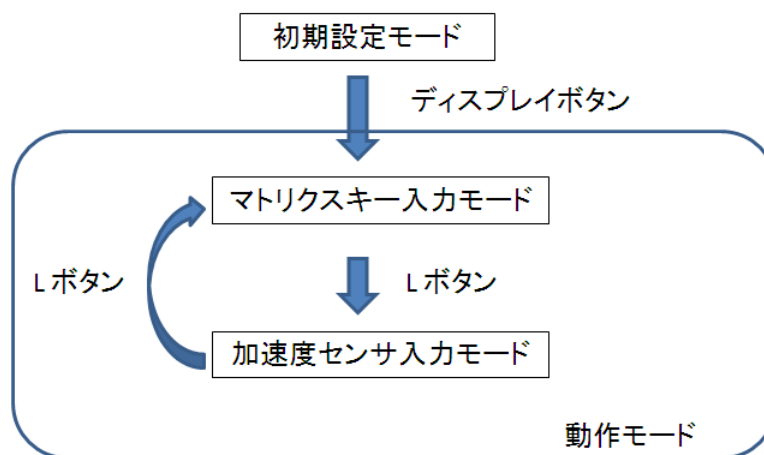


図 1 G&A Terminal の動作

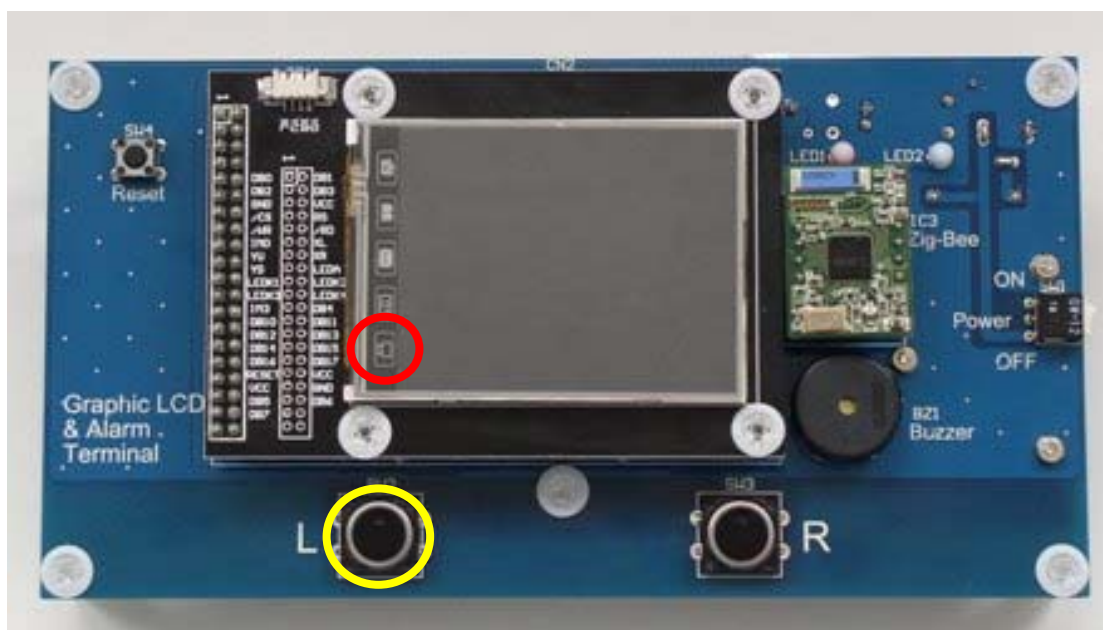


図 2 モードを切り替えるディスプレイボタン

《モード1》マトリクスキー入力モード

図3のようなマトリクス状の入力画面が表示されます。タッチペンで各数字や文字などをタッチすることで値を入力することができます。

それぞれのキーの意味は、下記のようになります。

0~9 キー	:	0~9 の数字
L キー	:	左回転
R キー	:	右回転
CLR キー	:	入力数字のクリア
RUN キー	:	データ送信

入力された文字は、ディスプレイ右上の表示領域に表示されます。回転方向は、表示領域の最上段に表示されています。デフォルトではR（右回転）となっています。RUN キーが押されると、図4のように表示領域の最上段にその値が表示されます。これらの値が ZigBee によってモータコントローラに送信されます。表示領域の最下段には ZigBee によってモータコントローラから受信された回転方向と回転数の数字が'N='の後に表示されます。

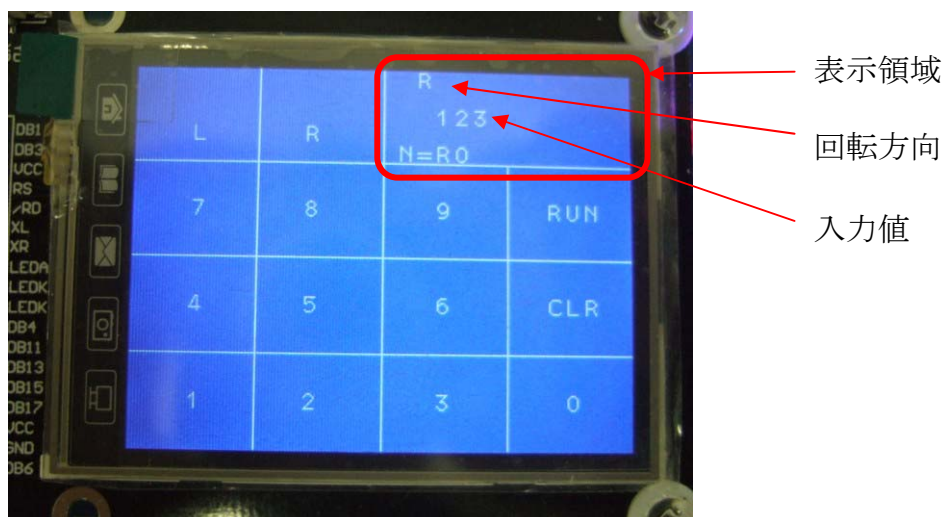


図3 マトリクスキー入力モード画面

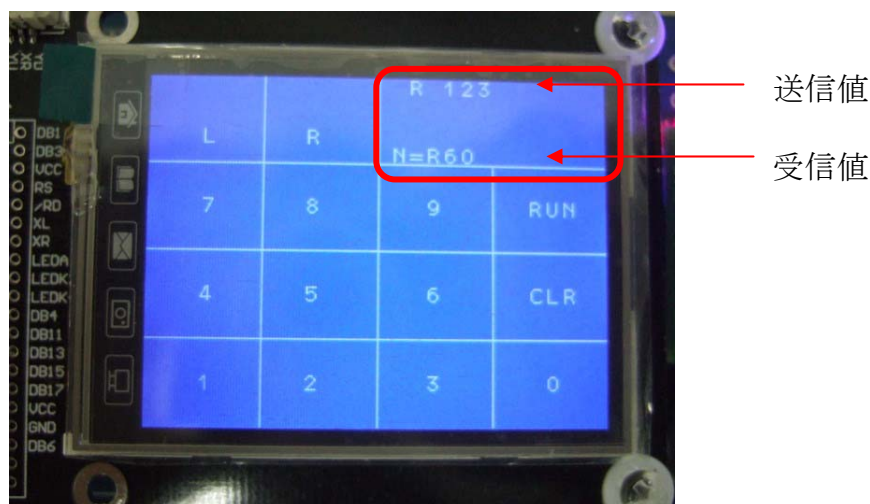


図4 RUN キーが押された時の画面

表示領域の入力値は，図5のように最大9桁まで入力できるようになっています．10桁目が入力されると強制的に表示領域の入力値はクリアされます．

また，RUN（送信）できる最大値は10000となっています．それ以上の値を入力してRUNキーをタッチすると図6のように表示領域の最上段に'Overflow'と表示され，データは送信されません．回転方向はRUNが押されるまでは，いつでも変更することができます．

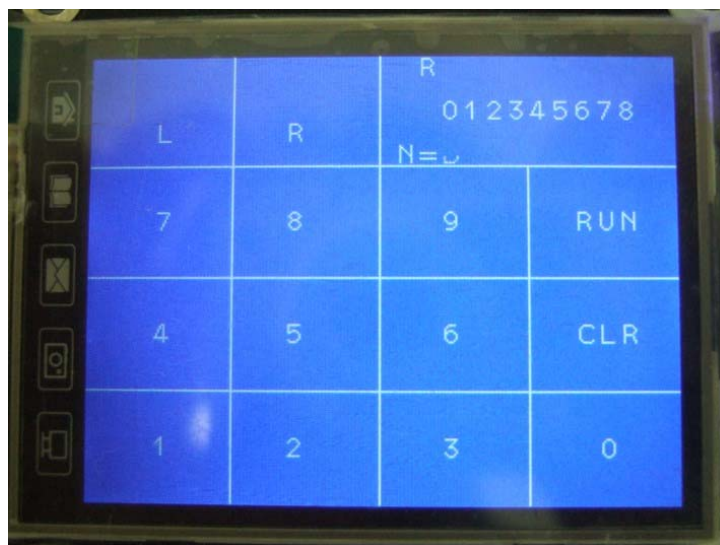


図5 最大となる入力文字数の一例



図6 入力値が10000を超えた場合

《モード2》加速度センサ入力モード

図7に示すように加速度センサを用いた入力画面が表示されます。画面左側には、加速度センサのx軸、y軸、z軸のA/D変換値が数値として表示されます。

G&A Terminal を机に置いた状態をまっすぐ上に持ち上げた G&A Terminal の姿勢を基本体勢とします。その時の入力値を0とし、現在の入力値を画面右上に表示します。基本体勢からゆっくりと右へ90度傾けていくとRの表示とともに入力値が約10000になるまで上昇します。同様に、基本体勢からゆっくりと左へ90度回転させていくとLの表示とともに入力値は約10000になるまで上昇します。

また、モード1と同様に、右側の最下段には ZigBee によって受信された回転方向と回転数の数字が'N='の後に表示されます。

モード2ではモード1とは違い、画面右上の入力値が定期的に送信されています。

また、基本体勢に対して、奥側を下に下げる動作をすると、入力値がロックされて、画面右側中央に"LOCK MODE"とロックされた値が表示されます。ロックを解除するには、奥側を上にする動作をすると解除され、ロック表示もなくなります。なお、デフォルトではロックモードにはなっていないため、図7の右画面中央の LOCK 表示は表示されていません。

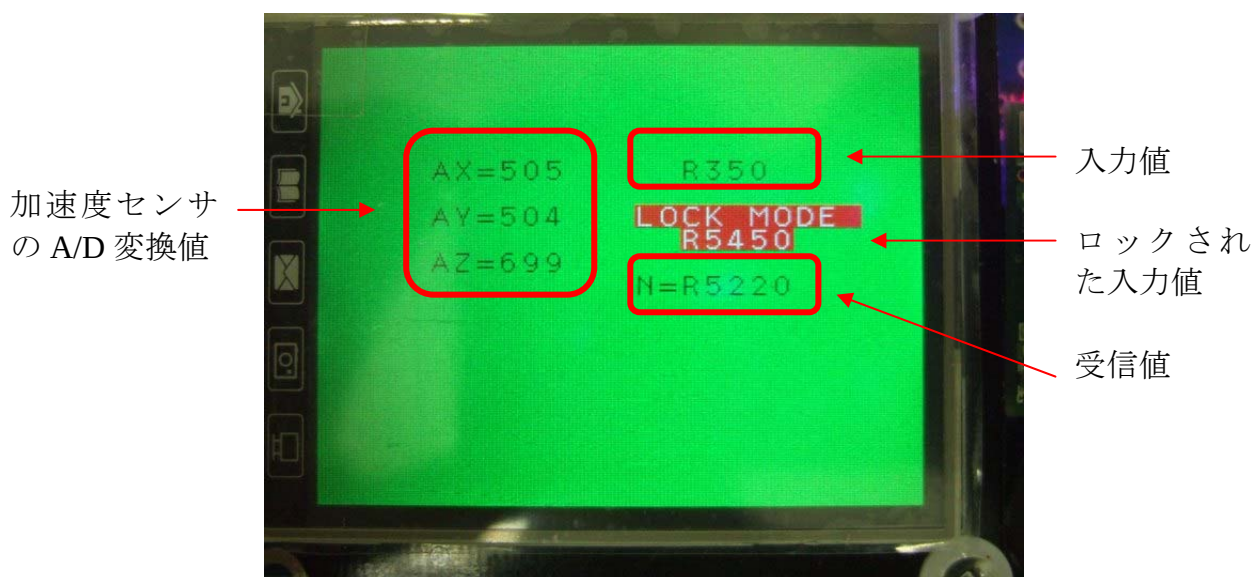


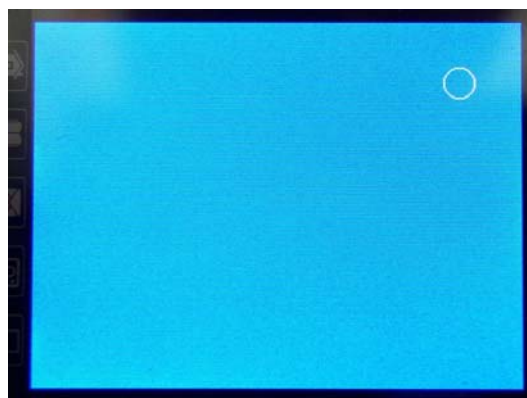
図7 加速度センサ入力モード画面

(2) G&A Terminal の初期設定モード (タッチパネルのキャリブレーション)

G&A Terminal のタッチパネルを利用するためには、初期設定 (キャリブレーション) が必要です。プログラムを書き込み直した場合は、その都度、初期設定を行う必要があります。

初期設定の方法を以下に示します。

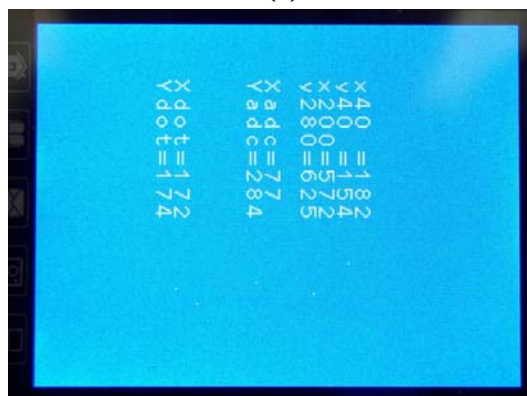
- ① G&A Terminal の “L” ボタンを押しながら、プログラムを起動させてください。
- ② 図 8(a)のように、画面右上に小さな円が現れます。円の中をタッチペンでタッチしてください。
- ③ 次に図 8(b)のように、画面左下に円が現れるので、同様にその円の中をタッチしてください。初期設定はこれで終了です。
- ④ この操作後に、タッチパネルをタッチすると、図 8(c)のように、補正されたその位置の A/D 変換値と、x 座標、y 座標の値が表示されます。
- ⑤ ディスプレイボタンをタッチすれば、初期設定画面を終了し、動作モードになります。



(a)



(b)

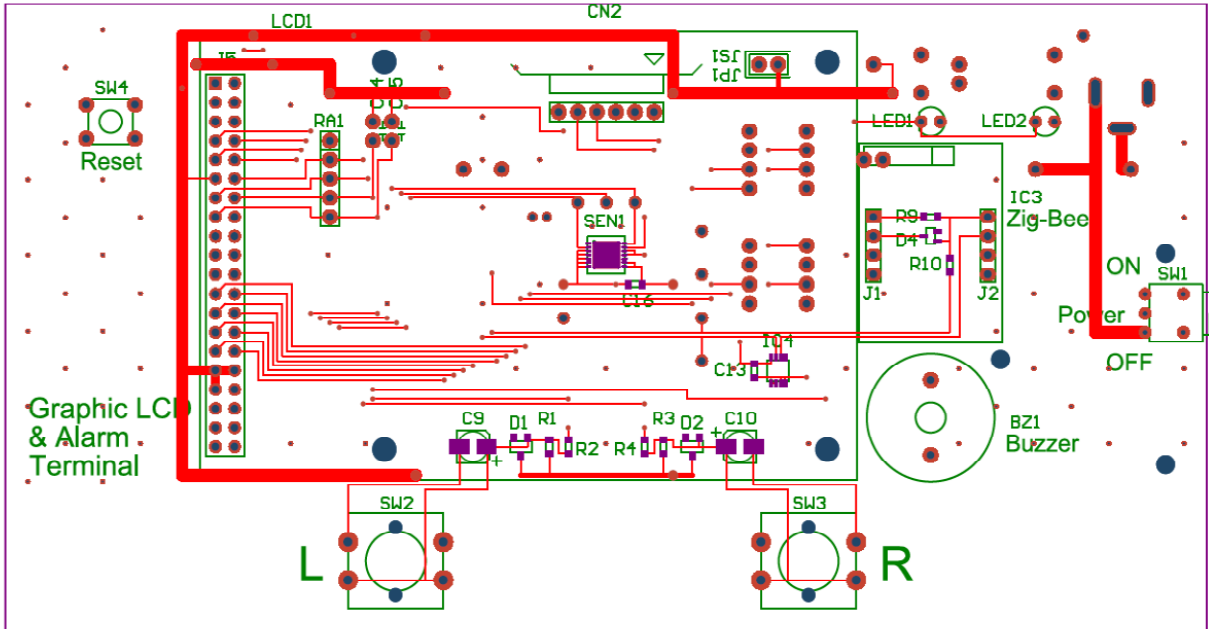


(c) タッチ位置の A/D 変換値と座標

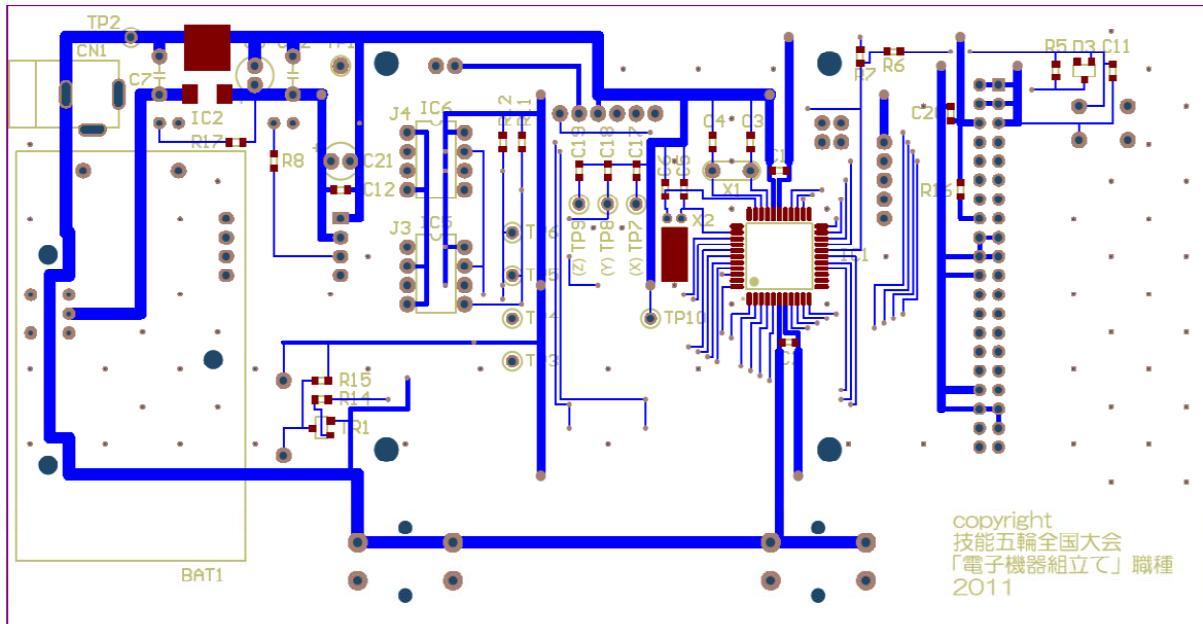
図 8 G&A Terminal の初期設定操作

Graphic LCD & Alarm Terminal Board 部品配置図

(表面)



(裏面)




```

1:  /*=====
2:  4620  第50回技能五輪全国大会「電子機器組立て」職種
3:  4620  競技 I
4:  4620  Title      : Color Graphic LCD表示プログラム
5:  4620  Terget PIC  : PIC18LF4620
6:  4620  Clock      : 10MHz
7:  4620  Compiler Ver : MCC18 Ver3.43
8:  4620  filename    : lcm240_v8.c
9:  4620  Date        : 2012.10.now
10: 4620=====
11: 4620          Copyright(C) 2012 H. Tamura, S. Miyazaki, Y. Yajima
12: 4620=====*/
13:
14: #include <p18f4620.h>
15: #include <timers.h>
16: #include <stdio.h>
17: #include <delays.h>
18: #include <portb.h>
19: #include <adc.h>
20: #include <i2c.h>
21: #include <usart.h>
22: #include "cglcd_lib_v8.h"
23: #include "touch_scr_lib.h"
24: #include "accel_ad_lib.h"
25: #include "usart_lib.h"
26: #include "kadai_xx.h"
27:
28: //OSC 10MHzx4=40MHz
29: #pragma config OSC = HSPLL
30: #pragma config PWRT = ON
31: #pragma config BOREN = OFF
32: #pragma config WDT = OFF
33: #pragma config MCLRE = ON
34: #pragma config PBADEN = OFF
35: #pragma config LVP = OFF
36:
37: #define L_Sw PORTBbits.RB6
38: #define Buzzer PORTBbits.RB5
39: #define delim 0xF0
40:
41: #define top_x 0
42: #define btm_x 240
43: #define top_y 0
44: #define btm_y 320
45:
46: char flag;
47: char rec_data[10];
48: unsigned char rec_count = 0;
49:
50: void init(void);
51:
52:
53: //-----
54: //割り込み
55: //高位割り込み
56: #pragma interrupt High_priority
57: #pragma code VectorHigh=0x08
58: void interrupt_VectorHigh(void)
59: {
60:     _asm
61:         goto High_priority
62:     _endasm
63: }
64:
65: #pragma code
66: void High_priority(void)
67: {
68:     if (PIR1bits.RCIF) {
69:         rec_data[rec_count++] = ReadUSART();
70:         if (rec_data[rec_count-1] == delim)
71:         {
72:             rec_data[rec_count-1] = '¥0';
73:             rec_count = 0;
74:         }
75:         PIR1bits.RCIF = 0;
76:     }
77: }
78:
79: /*-----*/
80:
81: void init(void)
82: {
83:     TRISA = 0xFF; //PortA:0-7 Input
84:     TRISB = 0xC0; //PortB:0-5 Output, 6,7 Input
85:     TRISC = 0x83; //PortC:2-6 Output, 0,1,7 Input
86:     TRISD = 0x00; //PortD:0-7 Output

```

```

87:
88: //AD変換初期設定
89: OpenADC(ADC_FOSC_16 & ADC_8_TAD & ADC_RIGHT_JUST,
90:         ADC_INT_OFF & ADC_CH0 & ADC_REF_VDD_VSS,
91:         ADC_6ANA);
92:
93: //I2C初期設定
94: OpenI2C(MASTER, SLEW_ON); //マスターモード
95: SSPADD = 0x18; //BRG 24 400kHz
96:
97: /*use USART*/
98: TRISCBits.TRISC7 = 1; /*USART RX*/
99: TRISCBits.TRISC6 = 0; /*USART TX*/
100:
101: PIR1bits.RCIF = 0; //受信割り込みフラグクリア
102: INTCONbits.PEIE = 1; //グローバル割り込み許可
103: INTCONbits.GIE = 1; //グローバル割り込み許可
104:
105: init_LCD();
106:
107: }
108:
109:
110: /*****
111: * タッチパネルのマトリックスキーの読み取り関数
112: *****/
113: char matlix_key_read(unsigned int *x, unsigned int *y)
114: {
115:     unsigned int xdot,ydot;
116:     char key;
117:
118:     key=0;
119:
120:     while(1){
121:         Touch_adr2(&xdot,&ydot); //タッチパネルの座標読取
122:         *x=xdot;
123:         *y=ydot;
124:         if(xdot<240 && xdot>=180 && ydot<320 && ydot>=240 ){
125:             key='1';
126:             break;
127:         }
128:         else if(xdot<180 && xdot>=120 && ydot<320 && ydot>=240 ){
129:             key='4';
130:             break;
131:         }
132:         else if(xdot<120 && xdot>=60 && ydot<320 && ydot>=240 ){
133:             key='7';
134:             break;
135:         }
136:         else if(xdot<240 && xdot>=180 && ydot<240 && ydot>=160 ){
137:             key='2';
138:             break;
139:         }
140:         else if(xdot<180 && xdot>=120 && ydot<240 && ydot>=160 ){
141:             key='5';
142:             break;
143:         }
144:         else if(xdot<120 && xdot>=60 && ydot<240 && ydot>=160 ){
145:             key='8';
146:             break;
147:         }
148:         else if(xdot<240 && xdot>=180 && ydot<160 && ydot>=80 ){
149:             key='3';
150:             break;
151:         }
152:         else if(xdot<180 && xdot>=120 && ydot<160 && ydot>=80 ){
153:             key='6';
154:             break;
155:         }
156:         else if(xdot<120 && xdot>=60 && ydot<160 && ydot>=80 ){
157:             key='9';
158:             break;
159:         }
160:         else if(xdot<240 && xdot>=180 && ydot<80 && ydot>=0 ){
161:             key='0';
162:             break;
163:         }
164:         else if(xdot<180 && xdot>=120 && ydot<80 && ydot>=0 ){
165:             key='C';
166:             break;
167:         }
168:         else if(xdot<120 && xdot>=60 && ydot<80 && ydot>=0 ){
169:             key='G';
170:             break;
171:         }
172:         else if(xdot<60 && xdot>=0 && ydot<240 && ydot>=160 ){

```

```

173:         key='R';
174:         break;
175:     }
176:     else if(xdot<60 && xdot>=0 && ydot<320 && ydot>=240) {
177:         key='L';
178:         break;
179:     }
180:     else{
181:         key=0;
182:         break;
183:     }
184: }
185: return key;
186: }
187:
188:
189: /*****
190: * マトリクスキーを押した箇所への枠表示関数
191: *****/
192: void pushed_sign(unsigned int x, unsigned int y, unsigned int color){
193:     unsigned int i,j;
194:     int k,l;
195:     unsigned int fontsize=12;
196:
197:     k=x/60;
198:     l=y/80;
199:
200:     //デフォルトの右上端は、結果表示の関係上、右下端へ変更
201:     if(k==0 && l==0) {
202:         k=3;
203:     }
204:
205:     for (j=0;j<3;j++) { // 上ライン
206:         Trans_Com_16(0x0020);Trans_Dat_16(0x0002+j+60*k); // x座標をセット
207:         Trans_Com_16(0x0021);Trans_Dat_16(0x004e+80*l); // y座標をセット
208:         Trans_Com_16(0x0022);
209:         for (i=0;i<76;i++) {
210:             Trans_Dat_16(color);
211:         }
212:
213:         Trans_Com_16(0x0020);Trans_Dat_16(0x0037+j+60*k); // 下ライン
214:         Trans_Com_16(0x0021);Trans_Dat_16(0x004e+80*l); // x座標をセット
215:         Trans_Com_16(0x0022); // y座標をセット
216:         for (i=0;i<76;i++) {
217:             Trans_Dat_16(color);
218:         }
219:     }
220:     for (j=0;j<56;j++) { // 左ライン
221:         Trans_Com_16(0x0020);Trans_Dat_16(0x0002+j+60*k); // x座標をセット
222:         Trans_Com_16(0x0021);Trans_Dat_16(0x004e+80*l); // y座標をセット
223:         Trans_Com_16(0x0022);
224:         for (i=0;i<3;i++) {
225:             Trans_Dat_16(color);
226:         }
227:
228:         Trans_Com_16(0x0020);Trans_Dat_16(0x0002+j+60*k); // 右ライン
229:         Trans_Com_16(0x0021);Trans_Dat_16(0x0004+80*l); // x座標をセット
230:         Trans_Com_16(0x0022); // y座標をセット
231:         for (i=0;i<3;i++) {
232:             Trans_Dat_16(color);
233:         }
234:     }
235: }
236:
237: /*****
238: * マトリクスキーによる入力関数
239: * (タッチパネル)
240: *****/
241: int matlix_input_func(void)
242: {
243:     unsigned int xdot,ydot,previous_xdot=0,previous_ydot=0;
244:     char key=0,previous_key=0;
245:     unsigned char cursor=0;
246:     char str[10]={0};
247:     char rot='R';
248:     unsigned long num=0;
249:
250:     //初期値データ送信
251:     rot_data_usart(rot,0);
252:
253:     CloseUSART(); //USART割込み禁止
254:
255:     flag=1;
256:
257:     //インターフェースの作成
258:     lcd_Clear(BLUE);

```

```

259:
260: lcd_Line(60, 0, 60, 320, WHITE); //横線 上
261: lcd_Line(120, 0, 120, 320, WHITE); //横線 中
262: lcd_Line(180, 0, 180, 320, WHITE); //横線 下
263: lcd_Line(60, 80, 240, 80, WHITE); //縦線 右
264: lcd_Line(0, 160, 240, 160, WHITE); //縦線 中
265: lcd_Line(0, 240, 240, 240, WHITE); //縦線 左
266:
267: printChar12(3, 17, '1', WHITE, BLUE);
268: printChar12(3, 12, '4', WHITE, BLUE);
269: printChar12(3, 7, '7', WHITE, BLUE);
270: printChar12(9, 17, '2', WHITE, BLUE);
271: printChar12(9, 12, '5', WHITE, BLUE);
272: printChar12(9, 7, '8', WHITE, BLUE);
273: printChar12(16, 17, '3', WHITE, BLUE);
274: printChar12(16, 12, '6', WHITE, BLUE);
275: printChar12(16, 7, '9', WHITE, BLUE);
276: printChar12(23, 17, '0', WHITE, BLUE);
277: printChar12(23, 12, 'C', WHITE, BLUE);
278: printStr12_rom(22, 12, "CLR", WHITE, BLUE);
279: printStr12_rom(22, 7, "RUN", WHITE, BLUE);
280: printChar12(3, 3, 'L', WHITE, BLUE);
281: printChar12(9, 3, 'R', WHITE, BLUE);
282: printChar12(15, 0, rot, WHITE, BLUE);
283:
284: PIE1bits.RCIE = 1; //USART割込み許可
285: OpenUSART(USART_TX_INT_OFF & USART_RX_INT_ON &
286:           USART_ASYNC_MODE & USART_EIGHT_BIT &
287:           USART_CONT_RX & USART_BRGH_HIGH, 255); //9600bps
288:
289:
290: //入力された文字列の送信用配列の作成
291: while(flag){
292:     while(flag && cursor<10){
293:         key=matlrx_key_read(&xdot, &ydot);
294:         if(key != 0){ //押された時
295:             pushed_sign(xdot, ydot, RED); //押された箇所に赤枠表示
296:             str[cursor]=key;
297:         }
298:         else{ //離した時
299:             pushed_sign(previous_xdot, previous_ydot, BLUE); //押された箇所の赤枠消去
300:         }
301:
302:         if((key != 0) && (key != previous_key)){ //入力が変わった時
303:             if(key == 'C'){ //CLRが押された
304:                 line_clear(16, 2, 10, BLUE); //入力文字列の画面消去
305:                 line_clear(17, 0, 10, BLUE); //RUN文字列の画面消去
306:                 cursor=0;
307:             }
308:             else if(key == 'G'){ //RUNが押された
309:                 line_clear(17, 0, 10, BLUE); //RUN文字列の画面消去
310:                 /******
311:                 /* 数字文字を数値に変換する関数 課題1
312:                 /******
313:                 str_to_num(str, &num); //文字列を数値に変換
314:                 /******
315:                 line_clear(16, 2, 10, BLUE); //入力文字列の画面消去
316:                 cursor=0;
317:                 if(num <= 10000){ //入力させる上限値
318:                     printNumber12(17, 0, num, WHITE, BLUE); //RUN(送信する)数値表示
319:                     //zigbee
320:                     rot_data_usart(rot, num);
321:                 }
322:                 else{
323:                     printStr12_rom(17, 0, "Overflow", WHITE, BLUE);
324:                 }
325:             }
326:             else if(key== 'L' || key == 'R'){ //LまたはRが押された
327:                 rot=key;
328:                 printChar12(15, 0, rot, WHITE, BLUE);
329:             }
330:             else{ //数字が押された
331:                 line_clear(17, 0, 10, BLUE); //RUN文字列の画面消去
332:                 printChar12(16+cursor, 2, key, WHITE, BLUE); //文字の表示
333:                 cursor++;
334:             }
335:         }
336:         previous_key=key;
337:         previous_xdot=xdot;
338:         previous_ydot=ydot;
339:
340:         line_clear(16, 4, 10, BLUE);
341:         printStr12_rom(14, 4, "N=", WHITE, BLUE);
342:         printStr12(16, 4, rec_data, WHITE, BLUE);
343:
344:         //ディスプレイボタンが押されたときの処理

```

```

345:         if ((L_Sw == 0) || (ydot > 300 && xdot > 190)) {           //Lスイッチまたはディスプレイスイッチで抜ける
346:             flag=0;
347:             break;
348:         }
349:     }
350:     line_clear(16, 2, 10, BLUE);
351:     cursor=0;
352: }
353: return 2;
354:
355: }
356:
357: /*****
358: * 加速度センサによる入力関数
359: *****/
360: int Accel_input_func(void)
361: {
362:     unsigned int adx, ady, adz;
363:     signed int rpm, rpmL;
364:     unsigned char lock=0;
365:     char rot='R', rotL='R';
366:
367:     CloseUSART();
368:
369:     delay_ms(500);
370:
371:     flag=1;
372:     lcd_Clear(GREEN);
373:
374:     OpenUSART(USART_TX_INT_OFF & USART_RX_INT_ON &
375:               USART_ASYNC_MODE & USART_EIGHT_BIT &
376:               USART_CONT_RX & USART_BRGH_HIGH, 255); //9600bps
377:
378:     while (flag) {
379:
380:         /*****
381:         * 各加速度センサの現在の値を表示する関数 課題2
382:         *****/
383:         accle_print(&adx, &ady, &adz);
384:         /*****
385:
386:         rpm=(512-adx)*50; //回転数の変更(左右傾き)
387:
388:         //回転方向の状態
389:         if (adx<512) {
390:             rot='R'; //右回転
391:         }
392:         else {
393:             rot='L'; //左回転
394:             rpm=-rpm;
395:         }
396:
397:         line_clear(15, 5, 10, GREEN);
398:         printChar12(16, 5, rot, BLACK, GREEN);
399:         printNumber12(17, 5, rpm, BLACK, GREEN);
400:
401:         //回転数のロックと解除
402:         if (ady<410) { //回転数のロック
403:             lock=1;
404:             rpmL=rpm;
405:             rotL=rot;
406:             printStr12_rom(14, 7, "LOCK MODE ", WHITE, RED);
407:             printChar12(16, 8, rotL, WHITE, RED);
408:             printNumber12(17, 8, rpmL, WHITE, RED);
409:         }
410:         if (ady>610) { //回転数のロック解除
411:             lock=0;
412:             line_clear(14, 7, 11, GREEN);
413:             line_clear(16, 8, 10, GREEN);
414:         }
415:
416:         //zegbeeで送信
417:         if (lock == 1) { //ロックされた時
418:             rot_data_usart(rotL, rpmL);
419:         }
420:         else { //ロックされていない時
421:             rot_data_usart(rot, rpm);
422:         }
423:
424:         line_clear(16, 10, 10, GREEN);
425:         printStr12_rom(14, 10, "N=", BLACK, GREEN);
426:         printStr12(16, 10, rec_data, BLACK, GREEN);
427:
428:         //Lボタンが押されたときの処理
429:         if (L_Sw == 0) {
430:             flag=0;

```

```

431:         break;
432:     }
433: }
434: return 1;
435: }
436:
437:
438: /*=====*/
439: void main(void)
440: {
441:     unsigned char mode;
442:     //入出力ポートの設定
443:     init();
444:
445:     lcd_Clear(WHITE);      //画面クリア
446:
447:     //タッチパネルのキャリブレーション
448:     if (L_Sw == 0){
449:         Touch_calib();
450:         delay_ms(250);
451:     }
452:     else {
453:         Touch_calib_read();
454:     }
455:     delay_s(1);
456:     mode = 1;
457:
458:     //メニュー選択* -----
459:     while(1){
460:         switch (mode) {
461:             case 1:
462:                 mode = matlix_input_func();    //マトリクスキー入力
463:                 break;
464:             case 2:
465:                 mode = Accel_input_func();    //加速度センサ入力
466:                 break;
467:             default:
468:                 /* 想定外 */
469:                 break;
470:         }
471:     }
472: }

```

関数一覧表

cglcd_lib_v8.h	LCD関数
	void init_LCD(void);
	void lcd_Clear(unsigned int color);
	void printStr12(int colum, int line, char *s, unsigned short color1, unsigned short color2);
	void printStr12_rom(int colum, int line, const rom char *s, unsigned short color1, unsigned short color2);
	void drawLine(int x1, int y1, int x2, int y2, int color);
	void drawRectangle(int x1, int y1, int x2, int y2, int color);
	void drawRectangleFull(int x1, int y1, int x2, int y2, int color);
	void lcd_Circle(int x0, int y0, int r, unsigned int color);
	void paint_area_init(void);
	unsigned int select_color(unsigned int px, unsigned int py, unsigned int prv_color);
	void paint_init(void);
	void lcd_Pixel(int Xpos, int Ypos, unsigned int color);
	void lcd_BigPixel(int Xpos, int Ypos, unsigned int color);
	void lcd_Line(int x1, int y1, int x2, int y2, unsigned int color);
	void lcd_Picture(void);
	void lcd_Char12(int colum, int line, unsigned char letter, unsigned short color1, unsigned short color2);
	void lcd_Str12(char colum, int line, char *s, unsigned short color1, unsigned short color2);
	void lcd_Str12_rom(char colum, int line, const rom char *s, unsigned short color1, unsigned short color2);
	void lcd_Number12(char colum, int line, int num, unsigned short color1, unsigned short color2);
	void Trans_Com_16(unsigned int data1);
	void Trans_Dat_16(unsigned int data1);
	void delay_us(unsigned int usec);
	void delay_ms(unsigned int msec);
	void delay_s(unsigned int sec);
	追加 void printChar12(int, int, unsigned char, unsigned short, unsigned short);
	追加 void printNumber12(char, int, int, unsigned short, unsigned short);
	追加 void line_clear(unsigned int x,unsigned int y,unsigned int z,unsigned int color)
	I2C 関数
	void DataTrans(unsigned char data);
	unsigned char Read_EE(unsigned char DevAddr,long Addr,unsigned char *data,unsigned char length);
accel_ad_lib.h	加速度センサ関数
	unsigned int Accel_xadc(void);
	unsigned int Accel_yadc(void);
	unsigned int Accel_zadc(void);
touch_scr_lib.h	タッチパネル関数
	unsigned int Touch_xadc(void);
	unsigned int Touch_yadc(void);
	unsigned char Touch_judg(void);
	void Touch_calib(void);
	void Touch_calib_read(void);
	void Touch_adr(unsigned int *,unsigned int *);
追加	void Touch_adr2(unsigned int *xdot,unsigned int *ydot);
usart_lib.h	zigbee用関数
	unsigned char zig_rec_1byte(void);
追加	void rot_data_usart(char, unsigned int){
kadai_xx.h	課題用関数
	void str_to_num();
	void accle_print();

部品請求用紙

選手番号：

選手名：

部品記号	部 品 名	定格・形式	数量

第50回技能五輪全国大会

「電子機器組立て」職種競技委員主査 殿

ギブアップ宣言

私は,

☐ 設計課題

☐ 回路図作成課題

☐ 基板設計課題

において、ギブアップを宣言します。(該当する課題の□に“レ”を記入すること)

ギブアップした課題において、採点が対象外となることを了承します。

2012年10月27日 ____時____分

選手番号 _____番

選手名 _____ (自署)

この欄は競技委員が記入しますので、何も書かないでください。

提供したもの	回路図	印刷物 電子ファイル	
	基板図	印刷物	